2026-9-18 Linux网络--TCP连接管理与异常诊断

- 1. TCP 连接管理与异常诊断
- 2. 连接的本质与内核表示
  - 2.1 什么是 TCP 连接
    - 2.1.1 四元组与状态
    - 2.1.2 先描述，再组织
  - 2.2 对应用可见的连接手段
- 3. 三次握手
  - 3.1 报文交换
    - 3.1.1 三次握手过程
    - 3.1.2 握手由内核完成
    - 3.1.3 第三次 ACK 与数据
  - 3.2 为什么必须三次
    - 3.2.1 双向可达性
    - 3.2.2 双方意愿
- 4. 握手队列与连接建立观察
  - 4.1 半连接队列与全连接队列
    - 4.1.1 两类队列
    - 4.1.2 backlog
  - 4.2 tcpdump 与 Wireshark 观察
    - 4.2.1 tcpdump 基础
    - 4.2.2 Wireshark 检查点
- 5. 连接建立异常与 RST
  - 5.1 端口未监听
  - 5.2 旧 SYN 与旧连接报文
  - 5.3 连接成功不等于业务成功
- 6. 数据阶段
- 7. 四次挥手与半关闭
  - 7.1 FIN 的含义
    - 7.1.1 标准状态变化
    - 7.1.2 同时关闭与 CLOSING
  - 7.2 close 与 shutdown
  - 7.3 捎带 FIN
- 8. TIME_WAIT 与 CLOSE_WAIT
  - 8.1 TIME_WAIT
  - 8.2 CLOSE_WAIT

    - 8.3 地址复用
- 9. 异常断线与保活
    - 9.1 进程退出和机器重启
    - 9.2 物理断网
    - 9.3 TCP keepalive
    - 9.4 应用层心跳
- 10. 连接诊断清单
- 11. 小结

1. TCP 连接管理与异常诊断

TCP 连接不是一条永久存在的物理线路，而是双方内核共同维护的一组状态、序号、窗口、计时器和缓冲区。三次握手建立共识，数据传输阶段持续维护状态，四次挥手确认双方都不再发送普通数据，异常断网则由 RST、keepalive 或应用层心跳处理。

2. 连接的本质与内核表示

2.1 什么是 TCP 连接

2.1.1 四元组与状态

TCP 连接由源 IP、源端口、目的 IP、目的端口四元组标识。双方还需要保存：

- 当前连接状态；
- 下一个发送序号和期望接收序号；
- 发送窗口、接收窗口和拥塞窗口；
- 发送、接收缓冲区；
- 重传、延迟应答和保活相关计时器；
- MSS、窗口扩大因子、SACK 等协议选项。

2.1.2 先描述，再组织

单个连接先由内核结构体描述，再通过哈希表、队列和链表组织。大量连接的管理最终转化为对这些数据结构的查找、插入、状态修改和删除。

应用通过文件描述符操作连接，但连接对象生命周期由内核维护。进程退出会关闭自己持有的描述符，连接仍可能进入 TIME_WAIT 等状态。

2.2 对应用可见的连接手段

客户端通过 `connect` 发起连接；服务端通过 `listen` 进入监听状态，再通过 `accept` 获取已经完成的连接。

`connect` 通常阻塞到握手完成。`accept` 不执行握手，只从全连接队列取出已经建立的连接并返回新的 `connfd`。

3. 三次握手

3.1 报文交换

3.1.1 三次握手过程

设客户端初始序号为 `x`，服务端初始序号为 `y`：

步骤	方向	报文	客户端状态	服务端状态
监听	-	无连接报文	CLOSED	LISTEN
1	C -> S	`SYN, seq=x`	SYN_SENT	LISTEN
2	S -> C	`SYN, ACK, seq=y, ack=x+1`	SYN_SENT	SYN_RCVD
3	C -> S	`ACK, ack=y+1`	ESTABLISHED	ESTABLISHED

双方发送的是完整 TCP 报文，只是相应标志位被置为 1。SYN 和 FIN 会消耗一个序号位置。

3.1.2 握手由内核完成

客户端调用 `connect` 触发握手，双方 TCP 层自主交换 SYN、SYN+ACK 和 ACK。连接可以在服务端应用尚未 `accept` 时已经完成。

`accept` 只是把内核已经建立的连接取到应用上下文，并返回新文件描述符。

3.1.3 第三次 ACK 与数据

第三次 ACK 表示客户端确认服务端的 SYN，连接随后进入 ESTABLISHED。某些实现允许第三次 ACK 附带少量应用数据，但这不是所有协议扩展和所有服务的统一保证。

3.2 为什么必须三次

3.2.1 双向可达性

握手需要验证双方都能发送和接收。前两次交换后，客户端可以判断自己的发送方向和服务端的 SYN+ACK 可达；服务端还需要收到第三次 ACK，才能确认自己的 SYN+ACK 真正到达客户端。

3.2.2 双方意愿

服务端处于 LISTEN 表示愿意接受连接，客户端发送 SYN 表示主动请求。双方完成报文交换后，才形成可靠的通信共识。

两次不足，三次已能验证双向网络和双方状态；继续增加次数不会提供新的必要信息。

4. 握手队列与连接建立观察

4.1 半连接队列与全连接队列

4.1.1 两类队列

- 半连接队列：保存已经收到 SYN、但尚未完成握手的请求，服务端常见状态为 `SYN_RECV`；
- 全连接队列：保存已经完成握手、等待应用 `accept` 的连接，状态为 `ESTABLISHED`。

`SYN_SENT` 是客户端等待 SYN+ACK 的状态，不是服务端半连接队列。

4.1.2 backlog

`listen(fd, backlog)` 的第二个参数与等待应用接管的连接数量有关。课堂实验将参数设为 2，观察到前三个连接进入全连接队列，后续连接停在 `SYN_RECV`。

`backlog + 1` 是该环境下的观察结果，不是跨内核版本固定公式。实际队列还受 `somaxconn`、SYN Cookie、内存和协议栈实现影响。

长期不 `accept` 会使队列耗尽，新的握手无法最终完成。全连接队列可以缓存少量已完成连接，提高服务器满载率，但不能替代及时处理连接。

4.2 tcpdump 与 Wireshark 观察

4.2.1 tcpdump 基础

```
sudo tcpdump -i any tcp
sudo tcpdump -i eth0 tcp
sudo tcpdump src host 192.168.1.100 and tcp
sudo tcpdump dst host 192.168.1.100 and port 8000
sudo tcpdump -i eth0 port 8000 -w tcp.pcap
tcpdump -r tcp.pcap
```

常用参数：

- `-i`：选择网络接口；
- `host/src host/dst host`：按地址过滤；
- `port`：按端口过滤；
- `-w`：写入 pcap 文件；
- `-r`：读取 pcap 文件；
- `-n`：禁止主机名反向解析。

4.2.2 Wireshark 检查点

过滤条件可以使用：

```
ip.addr == <server_ip>
tcp.port == 8000
```

握手时应观察 SYN、SYN+ACK、ACK 的序号与确认号。Wireshark 默认可能使用相对序号，因此 SYN 显示为 0 并不代表真实初始序号为 0。纯 ACK 有效载荷长度为 0，通常不消耗新的序号空间。

关闭连接时可对比 FIN、ACK 和合并的 FIN+ACK。网络中只出现三段报文，可能是某次 ACK 与 FIN 捎带合并，并不代表关闭语义变成握手。

5. 连接建立异常与 RST

5.1 端口未监听

客户端向没有监听者的端口发送 SYN，目标系统通常返回 RST，`connect` 失败，而不是建立 ESTABLISHED 连接。

5.2 旧 SYN 与旧连接报文

网络中可能残留旧 SYN 或旧连接数据。若接收端没有对应连接状态，可能回复 RST，要求发送方停止使用错误连接。

RST 是立即复位，不执行正常四次挥手。收到 RST 后应清理旧连接状态，重新建立连接或结束业务。

5.3 连接成功不等于业务成功

三次握手只证明 TCP 连接可用，不代表服务端进程已经完成业务处理。连接成功后，应用仍可能超时、返回错误、发送业务拒绝或主动关闭。

6. 数据阶段

连接进入 ESTABLISHED 后，双方可以同时发送和接收数据。ACK 与数据可以捎带，窗口在报文往返中持续更新。

应用看到的是 `read/write`；内核负责：

- 分段与组包；
- 序号排序和去重；
- 发送缓存和重传；
- 接收窗口和零窗口探测；
- 拥塞控制；
- 计时器和状态变化。

应用需要处理短读、短写、EOF 和错误码。`read` 返回 0 通常表示对端已经有序关闭写方向。

7. 四次挥手与半关闭

7.1 FIN 的含义

FIN 表示发送方不再发送新的普通数据。TCP 全双工，两个发送方向可以分别关闭。

7.1.1 标准状态变化

假设客户端主动关闭：

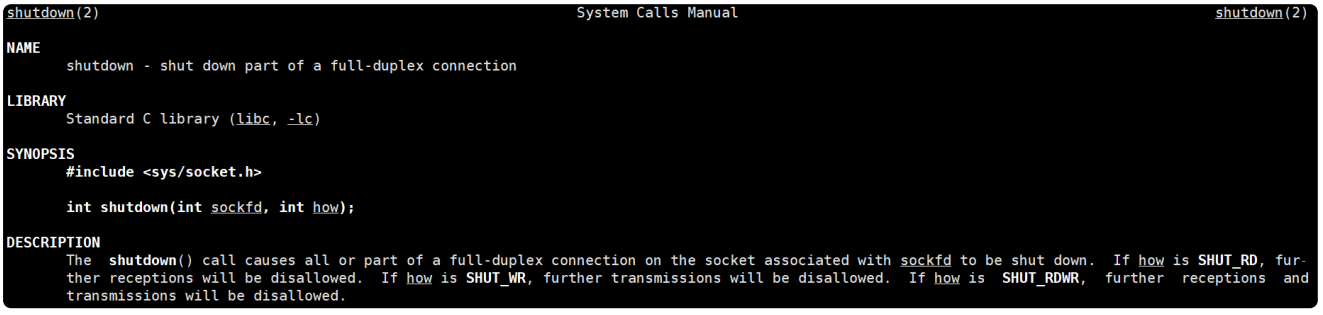
步骤	报文	客户端状态	服务端状态
1	客户端发送 FIN	FIN_WAIT_1	ESTABLISHED
2	服务端回复 ACK	FIN_WAIT_2	CLOSE_WAIT
3	服务端发送 FIN	FIN_WAIT_2	LAST_ACK
4	客户端回复 ACK	TIME_WAIT	CLOSED

连接结构体不会在发出 FIN 后立即释放。FIN_WAIT、CLOSE_WAIT、LAST_ACK 和 TIME_WAIT 都说明状态仍在推进。

7.1.2 同时关闭与 CLOSING

双方同时发送 FIN 时，双方可能先进入 `FIN_WAIT_1`。收到对方 FIN 并回复 ACK 后，自己的 FIN 尚未被确认，此时进入 `CLOSING`；收到自己的 FIN 的 ACK 后才进入 `TIME_WAIT`。

7.2 close 与 shutdown




```
shutdown(sockfd, SHUT_WR);    // 关闭写方向，仍可读
shutdown(sockfd, SHUT_RD);    // 关闭读方向
shutdown(sockfd, SHUT_RDWR);  // 两个方向都关闭
```

`close` 关闭文件描述符引用并推动连接关闭；`shutdown` 可以只关闭一个方向。客户端只下载时可以提前关闭写方向，但仍继续读取。

7.3 捎带 FIN

如果被动关闭方收到 FIN 时也没有剩余数据，可以把 ACK 和 FIN 合并发送。网络中表现为三段报文，但语义仍是两个方向的独立关闭。

8. TIME_WAIT 与 CLOSE_WAIT

8.1 TIME_WAIT

主动关闭方发送最后 ACK 后进入 TIME_WAIT。

```
ZL@VM-0-14-ubuntu:~$ cat /proc/sys/net/ipv4/tcp_fin_timeout
60
```

主要目的有两个：

1. 最后 ACK 丢失时，对端会重发 FIN，主动关闭方仍能再次回复；
2. 让旧连接的延迟报文在网络中消散，避免相同 IP 和端口的新连接误收。

传统模型把等待时间描述为 2MSL。Linux 的实际时长由内核实现决定，`/proc/sys/net/ipv4/tcp_fin_timeout` 不能简单地等同于每一个 TIME_WAIT 连接的生命周期。

大量短连接或服务端主动清理坏连接，可能造成较多 TIME_WAIT。应按连接模式、业务超时和复用策略综合排查。

8.2 CLOSE_WAIT

被动关闭方收到 FIN 并回复 ACK 后进入 CLOSE_WAIT。如果应用一直不关闭自己的 socket，连接会长期停留在这个状态。

大量 CLOSE_WAIT 通常表示应用没有正确 `close`，常见原因包括异常分支未释放、线程退出遗漏和回调未回收资源。

8.3 地址复用

服务端快速重启时，旧连接仍可能占用本地端口，导致 `bind` 失败。可以在监听套接字上设置：

```
int opt = 1;
setsockopt(
```

```
    listenfd,
    SOL_SOCKET,
    SO_REUSEADDR,
    &opt,
    sizeof(opt)
);
```

`SO_REUSEADDR` 只能在符合系统条件时复用地址，不能把所有连接状态无条件清除。

9. 异常断线与保活

9.1 进程退出和机器重启

进程正常退出会关闭文件描述符，内核通常能够发送 FIN。正常关机也可能先关闭进程，但强制断电、崩溃和设备瞬间失效时，双方可能没有机会完成挥手。

9.2 物理断网

一端断网后，远端只能观察到数据、ACK 或探测响应消失。网络恢复后如果旧端继续发送数据，对端可能返回 RST，旧端必须重新建立连接。

9.3 TCP keepalive

TCP keepalive 长期空闲后发送探测，连续无响应时关闭连接。它主要证明 TCP 路径和对端协议栈可达，不能证明业务服务正常。

Linux 通常需要应用显式启用 keepalive，默认空闲时间也可能很长，不适合秒级故障发现。

9.4 应用层心跳

应用层心跳定期发送轻量请求并检查正确响应，同时验证 TCP 连接、对端进程和业务处理能力。连续失败后关闭旧连接，进行带退避的重连。

心跳间隔、超时和失败次数应由业务实时性决定：过短浪费资源，过长延迟故障发现。

10. 连接诊断清单

现象	优先检查
connect 失败	服务是否 listen、目标 IP/端口、安全组、防火墙、RST
accept 得不到连接	是否在 accpet 前阻塞、全连接队列是否满、应用是否持续 accept
客户端已 ESTABLISHED，服务端 SYN_RECV	服务端队列或握手退出情况
大量 CLOSE_WAIT	应用是否未 close
大量 TIME_WAIT	短连接模式、主动关闭方、端口复用与连接池
长时间没有响应	网络路径、keepalive、应用层心跳、RST
抓包只看到三段关闭报文	ACK 与 FIN 可能捎带合并

11. 小结

TCP 连接管理的核心是状态和共识：

```
LISTEN
  -> SYN / SYN+ACK / ACK
  -> ESTABLISHED
  -> 双向数据与窗口更新
  -> FIN / ACK / FIN / ACK
  -> TIME_WAIT 或 CLOSING
  -> CLOSED
```

应用层 API 只是触发和观察这些状态；真正的握手、队列、状态转换、计时器和报文处理都在内核完成。及时抓包和检查 socket 状态，比只根据应用日志猜测连接问题更可靠。